

Conference Email System

Engineering Progress Report — Phases 0–4

AI-powered two-lane conference email management (FAQ auto-reply + human review). Generated June 30, 2026.

Phase 0 — Foundation

Project skeleton and the swappable-seam architecture that the rest of the system plugs into.

Backend scaffold (FastAPI app, typed config via pydantic-settings, async SQLAlchemy + Alembic, ORM models for emails / audit_logs / policy_documents), the six independently-replaceable modules (classifier, retriever, router, drafter, persistence, UI), and a Next.js frontend shell. Health check green; initial migration clean.

Phase 1 — Pipeline & API

The end-to-end processing pipeline and its HTTP surface.

Toy dataset (30 labeled emails) + knowledge base (45 policy chunks); a repository layer for all DB access; the classify → retrieve → route → draft pipeline plus an orchestrator; and the v1 API (ingest, queue, detail, approve, reroute, analytics) with a seed script. **16 tests passing.**

Phase 2 — Frontend

A complete, demo-ready operator interface.

React Query API client + hooks; a dark-mode design system (indigo accent); dashboard with live stats; an email queue split-pane review interface; the auto-replies table; an audit-log timeline; analytics charts; and a responsive app shell. All pages typed and served.

Phase 3 — Production Hardening & Intelligence

Five sub-phases (executed in the order 3E, 3C, 3D, 3A, 3B) that turned the MVP into a configurable, learnable, production-leaning system. Every capability is a config-flag swap; defaults are unchanged.

3E — Real Audit Endpoint

What was built. Replaced the audit stub with a real, paginated, filterable audit API reading directly from the audit_logs table: **GET /api/v1/audit** (filter by email_id / action / actor, with limit & offset) and **GET /api/v1/audit/{id}**.

Key decision. Added methods to the existing AuditRepository and mapped API fields (created_at ← timestamp, details ← extra_metadata) via Pydantic aliases instead of renaming DB columns — zero migration, fully backward compatible.

Intentionally not built. No write or delete endpoints — the audit trail is append-only by design; entries are created only by the pipeline and chair actions.

IN A MEETING

Every action the system or a chair takes is now permanently logged and queryable — full traceability for accountability and demos.

3C — PostgreSQL Migration Readiness

What was built. Added async (asyncpg) and sync (psycopg2) Postgres drivers, a SYNC_DATABASE_URL setting, a dialect-aware async Alembic env, and a zero-drift checkpoint migration. Switching to Postgres is one .env line plus *alembic upgrade head*.

Key decision. Kept SQLite as the default for dev and tests and made batch-ALTER SQLite-only, so Postgres migrations emit clean direct ALTERs and the URL passes through untouched.

Intentionally not built. Did not stand up a live Postgres server or run migrations against it (none in dev) — verified via the checkpoint migration and a model-vs-schema diff.

IN A MEETING

The system runs on lightweight SQLite today but is ready to flip to production Postgres with a single config line — no code changes.

3D — Local LLM Backend

What was built. Made the drafter provider-aware: MODEL_PROVIDER selects a hosted API, a local OpenAI-compatible endpoint (Ollama-style, MODEL_PROVIDER=local), or a deterministic fallback. Added a **GET /api/v1/health/model** status probe.

Key decision. Every provider path returns a draft and never raises; local errors degrade gracefully to a fallback with a logged warning, so a down model server can't break the pipeline.

Intentionally not built. No response streaming and no automatic provider failover — the surface stays simple; the provider is chosen by config, not runtime auto-selection.

IN A MEETING

We can run entirely on our own GPU with an open model for privacy and cost, or use a hosted API — switching is one config line, and the system stays up even if the model server is down.

3A — Trainable Classifier

What was built. A sentence-embedding + logistic-regression intent classifier that learns from labeled emails via **POST /api/v1/train/classifier** and is enabled with CLASSIFIER_BACKEND=trainable. Until trained, it auto-falls back to the keyword baseline.

Key decision. Kept the exact public classifier interface (a true drop-in), lazy CPU embedding load, and on-disk model artifacts — no churn for callers and no GPU requirement.

Intentionally not built. No deep fine-tuning or GPU training — a compact CPU model is sufficient at this data scale and keeps the loop fast and cheap.

IN A MEETING

The classifier can now learn from real labeled emails and improve over time, while safely falling back to rules until it has been trained.

3B — RL Bandit Router

What was built. An epsilon-greedy bandit ($\epsilon = 0.15$) that learns lane decisions per intent from chair feedback — approve rewards the chosen lane, reroute penalizes it. Enabled with ROUTING_STRATEGY=rl; win-rates at **GET /api/v1/analytics/rl-stats**.

Key decision. The learning layer sits on top of hard safety guards (sensitive intents and a low-confidence floor always escalate), so the bandit can never override safety.

Intentionally not built. No neural / contextual-feature policy — a per-intent bandit is interpretable and data-efficient for the current volume.

IN A MEETING

The router gets smarter every time a chair approves or reroutes an email, but it can never auto-reply to sensitive cases — safety rules always win.

Phase 3 summary

Sub-phase	What shipped	Config flag affected	Tests added
3E	Paginated, filterable audit endpoint (/api/v1/audit)	—	5
3C	PostgreSQL migration readiness (drivers, async Alembic, checkpoint)	SYNC_DATABASE_URL	0
3D	Local / hosted / fallback drafter + model health probe	MODEL_PROVIDER	4
3A	Trainable embedding classifier + train endpoint	CLASSIFIER_BACKEND	5
3B	RL bandit router + feedback loop + rl-stats	ROUTING_STRATEGY	6

Test count at end of Phase 3: 36 passing.

Phase 4 — Retrieval Quality & Evaluation

Two completed sub-phases that add semantic retrieval and a measurement harness — the tools to know whether the system is actually good.

4A — FAISS Vector Retrieval

What was built. A dense-vector retriever (sentence-transformer embeddings, all-MiniLM-L6-v2; FAISS IndexFlatIP with L2-normalized cosine) selectable via RETRIEVAL_BACKEND=faiss. Lazy index build, rebuild_index() for live reindex, new config FAISS_MODEL_NAME, and **GET /api/v1/retrieval/info**. BM25 is unchanged and still the default.

Key decision. Implemented as a flat module (not a subpackage) to avoid the import-shadowing bug removed in Phase 1C; loads documents from the DB via the policy repository using its own short-lived session, so retrieve() stays a true drop-in for BM25.

Intentionally not built. No approximate-nearest-neighbor index or GPU FAISS — exact flat search is instant at 45 chunks; kept CPU-only.

IN A MEETING

We can now match policies by meaning, not just keywords — and swap between keyword and semantic search with one config line, with the proven keyword method still the safe default.

4B — Eval Harness with Ground Truth

What was built. A labeled ground-truth set (data/eval/ground_truth.json — 40 emails, all 8 intents ×5, 15 FAQ / 25 human-review, 8 hard) and a component-level eval script (scripts/run_eval.py) that scores classifier + retriever

+ router: per-intent precision / recall / F1, routing accuracy, and retrieval hit-rate, written as a JSON report plus a console summary. CLI: --retrieval, --top-k, --output, --ground-truth, --verbose.

Key decision. Evaluate each component independently (not the full orchestrator) with no DB writes, so runs are fast, deterministic, and reproducible.

Intentionally not built. No FAISS eval against a seeded DB yet (BM25 baseline first) and no automated CI quality gate — this is a research tool for now.

IN A MEETING

We can now measure quality objectively and catch regressions — the harness already surfaced a concrete, fixable weakness (below).

BASLINE EVAL RESULTS (BM25, KEYWORD CLASSIFIER, TOP-K 3)

*Classification accuracy **95.0%** (macro F1 **0.950**) and retrieval hit-rate **97.5%** are strong. Routing accuracy is **77.5%**: human-review is perfect (25/25) but the FAQ lane is only **6/15**. The keyword classifier's confidence frequently falls below the 0.65 FAQ threshold, so genuine FAQ emails get escalated to humans — a clear, actionable signal to tune the threshold and/or train the classifier.*

Phase 4 summary

Sub-phase	What shipped	Config flag affected	Tests added
4A	FAISS dense retrieval + /retrieval/info (BM25 still default)	RETRIEVAL_BACKEND, FAISS_MODEL_NAME	6
4B	Eval harness + 40-email ground truth + JSON reports	—	5

Test count at end of Phase 4B: 47 passing.

WHAT COMES NEXT

Tune the FAQ confidence threshold (directly addresses the 6/15 finding); run the eval with the FAISS backend against a seeded database; Dockerize for deployment; and prepare the live demo.